

Cluster-by-time HDFE in `did_multiplegt_dyn`

Adding a focused option for FEs that are constant across groups within a time period

Worked example on `data_test.csv`: `ac_uq_id #monthyear`, a grid-cell panel where each grid (`unique_small_grid_id`) belongs to a single `ac_uq_id`.

1. The structure we want to absorb

Dataset (`data_test.csv`)

Panel at the (group, time) level:

- `unique_small_grid_id` -> group id `g`
- `monthyear` -> time id `t`
- `ac_uq_id` -> coarser group attribute, maps each grid to a single value for all `t` (time-invariant at the group level)

Goal. Absorb `ac_uq_id #monthyear` fixed effects -- one intercept per (cluster, time) cell -- alongside the time FE that `did_multiplegt_dyn` already partials out.

The defining feature: the absorbing variable is constant over groups inside the cluster and over the time periods we want to allow it to vary. Formally,

```
a_g = ac_uq_id_g in {1, ..., A}    (time-invariant)
absorb dimension: (a_g, t) in {1, ..., A} x {1, ..., T}
```

This is the same structural property that `trends_nonparam` already exploits with one variable; the proposed option generalizes it to multiple such variables and routes the demeaning through `reghdfe/hdfe`.

2. Why this case deserves its own option

- The variance derivation in dCdH (2024) is built for the projection of Delta Y and Delta X_k onto time FE (optionally x a time-invariant stratum). Cluster x time FEs of the form `a_g x t` are a linear extension of that subspace; the algebra goes through verbatim.
- Section 1.4 of the companion paper already discusses `trends_nonparam`. We are not opening a new derivation -- we are saying 'allow multiple such strata, and use `reghdfe` as the implementation back-end.'
- Performance: one-way demean by `(a_g, t, d)` is the same `bys ... gegen pattern` already at L931-934. Two-way absorb (`state#year industry#year`) is what forces us to call `reghdfe`.
- Restricted scope keeps validation simple. The option does NOT accept slope FEs, time-varying categoricals, or unit FEs at a finer level than the group. Only 'time-invariant group attribute x time'.

3. Proposed syntax

```
did_multiplegt_dyn Y G T D, effects(L) [other options] ///
                    cluster_time_fe(varlist)
```

Semantics:

- Each variable $a^{(1)}, \dots, a^{(P)}$ in the varlist must be time-invariant at the group level (checked at runtime).
- For each $a^{(p)}$, the option adds the FE set $\{1\{a^{(p)}_g = a_0\} \cdot 1\{t = t_0\}\}$ to the projector used to residualize Delta Y and each Delta X_k , separately per baseline level d.
- Time FE and (optionally) trends_nonparam stay where they are; the new option just appends more dummies to F_d .
- If the user passes a variable that is not time-invariant at the group level, the command exits with a descriptive error.

Concrete call on `data_test.csv`

```
import delimited using "data_test.csv", clear
did_multiplegt_dyn Y unique_small_grid_id monthyear D, ///
    effects(4) placebo(2) controls(X1 X2)           ///
    cluster_time_fe(ac_uq_id)                       ///
    cluster(ac_uq_id)
```

4. Stage 1 -- the one stage that actually changes

Stage 1 partials a projector P_{F_d} out of Delta Y and each Delta X_k , then runs OLS on the residuals to recover θ_d . All we change is the projector.

Augmented FE space F_d

```
F_d = span{
    1{t = t0},                                // time FE (always)
    1{s_g = s0} . 1{t = t0},                 // optional trends_nonparam
    1{a^{(p)}_g = a0} . 1{t = t0}, p=1..P    // cluster_time_fe (new)
}, restricted to S_d (control sample).
```

The weighted projector $P^W_{F_d} = Z(Z'WZ)^+ Z'W$ and its annihilator $M^W_{F_d} = I - P^W_{F_d}$ project onto F_d in the weighted inner product $W_d = \text{diag}(N_{gt})$.

Bottom line

The MATH is identical to the existing Stage 1. Replace every appearance of 'demean by time FE within S_d ' with 'demean by F_d within S_d '.

The four substitutions, line by line

- (1) Residualized FD of each control [L941]: $\text{resid}_{X_k} = \sqrt{N_{gt}} * [M^W_{F_d} \cdot \Delta X_k]$.
- (2) Outcome FD residual [L955]: $\text{diff}_{y_w} = \sqrt{N_{gt}} * [M^W_{F_d} \cdot \Delta Y]$. (Old code did not partial time FE out of DY here; with multi-way absorb, FWL requires both sides to be demeaned by the same projector.)
- (3) Per-control score [L958]: $\text{prod}_{X_k} = N_{gt} * [M^W_{F_d} \cdot \Delta X_k]$.
- (4) Fitted FE-only mean of DY [L984-996]: $E_{y_hat_gt}(d) = [P^W_{F_d} \cdot \Delta Y]$ (via `reghdfe + residuals()`).

What does NOT change in Stage 1

- matrix accum overall_XX = diff_y_wXX `mycontrols_XX' at L1018: same call.
- $\theta_d = M_d^{-1} q_d$ at L1034: same formula.
- $Den_d^{-1} = M_d^{-1} \cdot N_d^{tot} \cdot G$ at L1052: same formula.
- Singular-cell guard at L1042-1090: same logic; just check $rank(Z_d) = e(df_a)$ from reghdfe.

5. Stages 2 and 3 -- no math change, just plug in new θ_d

Zero code changes downstream

Stages 2 and 3 consume `coefs_sq_d'XX`, `inv_Denom_d'XX`, `prod_X*_Ngt_XX` and `E_y_hat_gt_int_d'XX`. Each is rebuilt from the augmented projector, so the downstream blocks at L4582-4689 (effects), L5145-5184 (placebos), and L4905-4954 (variance correction) automatically use the new HDFE structure. No line in those blocks needs to be touched.

For reference, the only points where Stage-1 outputs are read:

- L4679 -- $\Delta Y^{(l)} := \theta_d[k] \cdot \Delta X_k^{(l)}$.
- L5177 -- same line for the placebo long-difference.
- L4647, L4664 -- $(\Delta Y - E_{y_hat_gt}(d))$ inside the score $in_sum_{\{k,d\}}$.
- L4929 -- $[Den_d^{-1}] \cdot in_sum_d_{\{j,k\}}$, the matrix-vector product in the variance correction.
- L4938 -- subtract $\theta_d[j]$ from the influence-function block.
- L4912-4946 -- $prod_X_k$ inside the $in_sum / U^{var,X}$ computation.

Every consumer is satisfied as long as the Stage-1 outputs reflect the augmented projection.

6. Stata patch using reghdfe / hdfe

Piece A -- Syntax and dependency check (L53, L66)

```

* L53 -- add the option:
syntax varlist(min=4 max=4 numeric) [if] [in] [, ///
/* ... existing options ... */          ///
cluster_time_fe(varlist)]

* After the gtools check at L65-73, add:
if "`cluster_time_fe'" != "" {
    qui cap which reghdfe
    if _rc {
        di as error "cluster_time_fe() requires reghdfe. Run: ssc install reghdfe"
        exit 198
    }
    qui cap which hdfc
    if _rc { di as error "... requires hdfc. Run: ssc install hdfc" ; exit 198 }

    * Each variable must be time-invariant at the group level.
    foreach v of varlist `cluster_time_fe' {
        tempvar sd_v
        bys `2': egen `sd_v' = sd(`v')
        sum `sd_v'
        if r(max) > 0 {
            di as error "cluster_time_fe(): `v' is not time-invariant at the group level."
            exit 198
        }
        drop `sd_v'
    }
}

```

Piece B -- Replace L919-961 (demean step)

```

* Build the FE set F_d to absorb.
local fe_set "time_XX"
if "`trends_nonparam'" != "" {
    capture drop trends_nonparam_temp_XX
    gegen trends_nonparam_temp_XX = group(`trends_nonparam')
    local fe_set "`fe_set' i.trends_nonparam_temp_XX#i.time_XX"
}
foreach v of varlist `cluster_time_fe' {
    local fe_set "`fe_set' i.`v'#i.time_XX"
}

local mycontrols_XX ""
local count_controls = 0
foreach var of varlist `controls' {
    local ++count_controls
    cap drop resid_X`count_controls'_time_FE_XX
    cap drop prod_X`count_controls'_Ngt_XX
    gen resid_X`count_controls'_time_FE_XX = .
    gen prod_X`count_controls'_Ngt_XX = .
    local mycontrols_XX "`mycontrols_XX' resid_X`count_controls'_time_FE_XX"
}
cap drop diff_y_wXX
gen diff_y_wXX = .

levelsof d_sq_int_XX, local(levels_d_sq_XX)
foreach l of local levels_d_sq_XX {
    tempvar smp
    gen byte `smp' = (ever_change_d_XX == 0          ///
                    & diff_y_XX != .                ///
                    & fd_X_all_non_missing_XX==1    ///
                    & d_sq_int_XX == `l'            ///
                    & time_XX < F_g_XX)

    local dvars "diff_y_XX"
    forvalues k = 1/`count_controls' { local dvars "`dvars' diff_X`k'_XX" }
    qui hdfc `dvars' [aw=N_gt_XX] if `smp', ///
        absorb(`fe_set') generate(_resXX_)

    replace diff_y_wXX = sqrt(N_gt_XX) * _resXX_diff_y_XX ///
        if d_sq_int_XX == `l' & `smp' == 1
    forvalues k = 1/`count_controls' {
        replace resid_X`k'_time_FE_XX = sqrt(N_gt_XX) * _resXX_diff_X`k'_XX ///
            if d_sq_int_XX == `l' & `smp' == 1
        replace resid_X`k'_time_FE_XX = 0 if missing(resid_X`k'_time_FE_XX) ///
            & d_sq_int_XX == `l'
        replace prod_X`k'_Ngt_XX = sqrt(N_gt_XX) * resid_X`k'_time_FE_XX ///
            if d_sq_int_XX == `l'
        replace prod_X`k'_Ngt_XX = 0 if missing(prod_X`k'_Ngt_XX) ///
            & d_sq_int_XX == `l'
    }
    drop _resXX_* `smp'
}

```

Piece C -- Replace L984-996 (FE-only fitted value)

```
foreach l of local levels_d_sq_XX {
    cap drop E_y_hat_gt_int_`l'_XX
    local controlsXX ""
    forvalues k = 1/`count_controls' { local controlsXX "`controlsXX' diff_X`k'_XX" }

    cap reghdfe diff_y_XX `controlsXX' [aw=N_gt_XX] ///
        if d_sq_int_XX == `l' & time_XX < F_g_XX, ///
        absorb(`fe_set') noconstant residuals(_res_DY_l_XX)
    scalar rc_XX = _rc

    if rc_XX == 0 {
        gen E_y_hat_gt_int_`l'_XX = diff_y_XX - _res_DY_l_XX ///
            if d_sq_int_XX == `l' & time_XX < F_g_XX
        drop _res_DY_l_XX
        scalar df_a_`l'_XX = e(df_a)
    }
    else {
        drop if d_sq_int_XX == `l'
        noi di "Baseline Treatment Level `l' dropped because of insufficient observations."
    }
}
```

The unchanged tail (L1018-1098)

matrix accum at L1018, $\theta_d = M_d^{-1} q_d$ at L1034, $\text{Den}_d^{-1} = M_d^{-1} \cdot N_d^{\text{tot}} \cdot G$ at L1052, the singular-cell guard at L1042-1090, and the dropping of cells at L1094-1096 run verbatim on the new residuals. No code change.

7. End-to-end example on data_test.csv

```
* Path on the user's machine:
* /Users/anzony.quisperojas/Documents/GitHub/did_multiplegt_dyn/Stata/data_test.csv

import delimited using "data_test.csv", clear

* Confirm the cluster-time FE candidate is time-invariant per group:
bys unique_small_grid_id: egen sd_ac = sd(ac_uq_id)
assert sd_ac == 0 | missing(sd_ac)
drop sd_ac

did_multiplegt_dyn Y unique_small_grid_id monthyear D, ///
    effects(4) placebo(2) ///
    controls(X1 X2) ///
    cluster_time_fe(ac_uq_id) ///
    cluster(ac_uq_id)
```

What this estimates

For each baseline-treatment level d:

```
theta_d_hat = argmin_{theta, alpha^{(d)}}
  sum_{(g,t) in S_d} N_{gt} . [ Delta Y - sum_k theta_k Delta X_k - alpha^{(d)}_{a,g,t} ]^2
```

where $\alpha^{(d)}_{a,t}$ is an `ac_uq_id`-by-monthyear FE, allowed to differ by baseline cell `d`.

The event-study estimators then use $Y_{\{g,t\}} - Y_{\{g,t-l\}} - (\Delta X^{(l)}_{\{g,t\}})' \theta_{d_hat}$ as today, and the variance correction $U^{var,X}$ uses the Den_d^{-1} built off this projection.

8. Sanity check: equivalence with `trends_nonparam`

Regression test before merging: passing a single `a^(1)` via `cluster_time_fe(ac_uq_id)` should give numerically identical event-study coefficients to the current command run with `trends_nonparam(ac_uq_id)`, because `F_d` collapses to what `trends_nonparam` already builds at L991.

```
* OLD:
did_multiplegt_dyn Y unique_small_grid_id monthyear D,    ///
  effects(4) controls(X1 X2) trends_nonparam(ac_uq_id) ///
  cluster(ac_uq_id)
matrix b_old = e(b)
matrix V_old = e(V)

* NEW:
did_multiplegt_dyn Y unique_small_grid_id monthyear D,    ///
  effects(4) controls(X1 X2) cluster_time_fe(ac_uq_id) ///
  cluster(ac_uq_id)
matrix b_new = e(b)
matrix V_new = e(V)

matrix diff_b = b_old - b_new
matrix diff_V = V_old - V_new
matrix list diff_b
matrix list diff_V
```

Pass = max abs diff below $\sim 1e-8$. Extending to multi-FE (`cluster_time_fe(ac_uq_id another_cluster)`) has no `trends_nonparam` counterpart, but the FWL argument and the per-baseline-cell structure carry through.

9. Edge cases

- Group nested inside cluster. If `unique_small_grid_id` is finer than `ac_uq_id`, `ac_uq_id` is the cluster. If they are 1-to-1, `ac_uq_id#monthyear` reduces to `(g,t)` cells with one obs -- singularity guard fires.
- Singleton (cluster, year) cells. Use `reghdfe's keepsingletons` option; track `e(num_singletons)` for diagnostics.
- Missing values in `cluster_time_fe` variables. Drop the observation with a one-line warning.
- Interaction with `trends_nonparam`. The two options compose: `F_d` just stacks. `reghdfe` handles collinearity between `a_g x t` and a stratum `s_g x t` that overlaps with `a_g` via its rank-deficiency detector.
- Bootstrap and `by_path()`. Propagate `cluster_time_fe('cluster_time_fe')` into the inner-call locals at L1337, L2030, L2033.

10. Summary

The proposal in one paragraph

Add an option `cluster_time_fe(varlist)` that takes time-invariant group attributes. Each is interacted with time and appended to the FE space `F_d` used to residualize Delta Y and Delta X_k inside the control sample for each baseline level d. The residualization is performed by `hdfe` (one call per baseline cell, demeaning Delta Y and every Delta X_k at once). The FE-only fitted value `E_y_hat_gt_int`d'_XX` is rebuilt by `reghdfe ..., absorb(...) residuals()` and stored under the existing name. Nothing else changes: `matrix accum, theta_d, Den_d^{ -1}`, the long-difference residualization at L4679, and the variance correction at L4905-4954 all consume the rebuilt Stage-1 objects and produce correct event-study estimates and variances by Frisch-Waugh-Lovell and the existing companion-paper derivation.